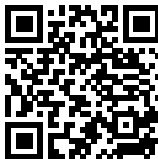


Data Structures & Algorithms @ Furpocalypse 2025

The Midnight Dimension

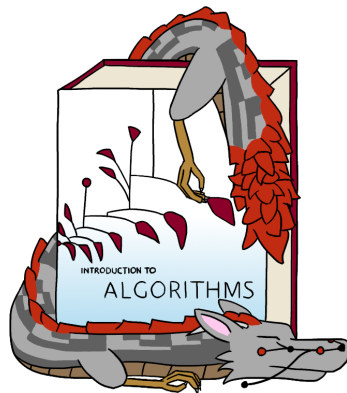
Bunsen Bitti, Unsigned Long

November 2, 2025



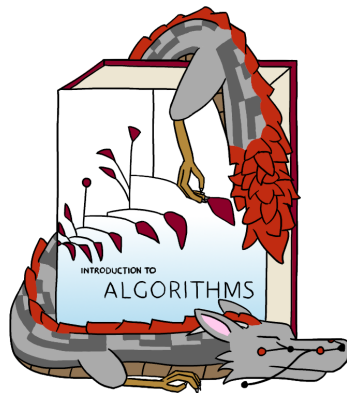
Overview

- This is a comedic/informative look at overlaps between furry and computer science
 - This is not a rigorous lecture
 - There will be puns ^w^



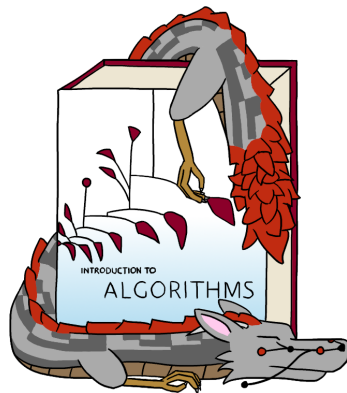
Overview

- This is a comedic/informative look at overlaps between furry and computer science
 - This is not a rigorous lecture
 - There will be puns ^w^
- Don't worry if our jokes don't make sense!
 - We've been coding for most of our lives
 - If you're confused, that's on us
 - We're out of touch with reality



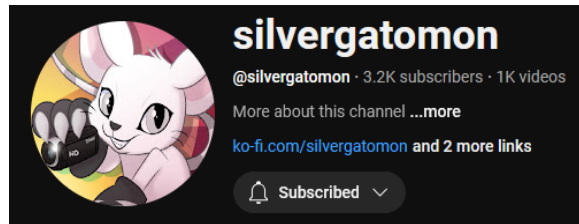
Overview

- This is a comedic/informative look at overlaps between furry and computer science
 - This is not a rigorous lecture
 - There will be puns ^w^
- Don't worry if our jokes don't make sense!
 - We've been coding for most of our lives
 - If you're confused, that's on us
 - We're out of touch with reality
- If you are curious about anything in these slides, feel free to chat with us!



How I Found the Fandom (Unsigned Long)

- Found furies through con videos



How I Found the Fandom (Unsigned Long)

- Found furies through con videos
- Found some eastern dragons with the word “Long” in their name



Tien Long

How I Found the Fandom (Unsigned Long)

- Found furies through con videos
- Found some eastern dragons with the word “Long” in their name
- Thought about “long” for too long

lóng
龙

How I Found the Fandom (Unsigned Long)

- Found furies through con videos
- Found some eastern dragons with the word “Long” in their name
- Thought about “long” for too long
- Realized it’s in my code

short	
short int	Short signed integer type. Capable of containing at least the $[-32\,767, +32\,767]$ range. ^{[3][a]}
signed short	
signed short int	
unsigned short	Short unsigned integer type. Contains at least the $[0, 65\,535]$ range. ^[3]
unsigned short int	
int	
signed	
signed int	Basic signed integer type. Capable of containing at least the $[-32\,767, +32\,767]$ range. ^{[3][a]}
unsigned	
unsigned int	Basic unsigned integer type. Contains at least the $[0, 65\,535]$ range. ^[3]
long	
long int	Long signed integer type. Capable of containing at least the $[-2\,147\,483\,647, +2\,147\,483\,647]$ range. ^{[3][a]}
signed long	
signed long int	
unsigned long	Long unsigned integer type. Capable of containing at least the $[0, 4\,294\,967\,295]$ range. ^[3]
unsigned long int	

How I Found the Fandom (Unsigned Long)

- Found furies through con videos
- Found some eastern dragons with the word “Long” in their name
- Thought about “long” for too long
- Realized it’s in my code
- Pun was too good to do nothing with, so I made a fursona



How I Found the Fandom (Bunsen)

- FNAF made me a bunny-obsessed furry back in 6th grade



artist: thepipefox (FurAffinity)

How I Found the Fandom (Bunsen)

- FNAF made me a bunny-obsessed furry back in 6th grade
- Started in algorithms, moved to hardware and performance engineering



artist: @Alextheyellowthing (Telegram)

How I Found the Fandom (Bunsen)

- FNAF made me a bunny-obsessed furry back in 6th grade
- Started in algorithms, moved to hardware and performance engineering
- Made an FPGA-controlled protogen for a final project because why not

ProtoFacer: FPGA-Driven Interactive Protogen Head

*Note: This is a partially redacted version (to respect Bunsen's privacy) of the original report submitted for MIT's 6.2550 Digital Systems Laboratory course.

1st Bunsen Bitti

Department of Electrical Engineering and Computer Science
Massachusetts Institute of Technology
Cambridge, MA, USA

Abstract—We present a design for a Protogen Head controlled by FPGA fabric. Using an inward-facing OV5640 camera inside the helmet, the Protogen Head reconstructs the cosplayer's face based on real-time video input. The FPGA processes 25 frames-per-second video input from the camera, (limited primarily by electromagnetic interference concerns).

Experimental testing shows that the ProtoFacer is able to capture blinks and mimic mouth movement during speech, all while using under 400mW of power (excluding external power to the LED boards) and 1.5 MiB of block RAM. This makes it ideal for use in limited memory/power environments.

Index Terms—Digital systems, Field programmable gate arrays, Image processing.

I. INTRODUCTION

Protogen heads are a type of mascot head which display a face using multiple LED boards behind a limited visor. These allow for a greater range of expressions for the cosplayer to choose from since these displays can flexibly display many different facial expressions without mechanical components. Ideally, the displayed face should be representative of the cosplayer's face, allowing for common facial movements such as blinking and smiling to be expressed. However, most models construct faces from a fixed set of pre-determined expressions stored on the controller's memory.

The goal of the ProtoFacer project is to overcome this limitation by leveraging the parallelism of the FPGA fabric to implement a face reconstructor from real-time video input. This presents a number of design challenges:

- The system needs to fit within the limited space of a protogen head (which is no larger than a 6-inch box) and leave enough space for the camera to see the cosplayer's face.
- The system should use a small amount of power; it would be difficult for cosplayers to carry around large power

- Image Processing Pipeline (Section III-B)
- Expression Recognizer (Section IV-A)
- Face Image Buffers (Section V-A)
- Face Reconstructor (Section V-B)
- HUB75 Driver (Section VI-B)
- Power Modulator (Section VI-C)
- HUB75 LED Boards

We evaluate the performance of the ProtoFacer based on its resource usage in Section VII. We then end the report with a discussion of the system's limitations and future work in Section VIII.

II. PHYSICAL CONSTRUCTION

A. Head Design

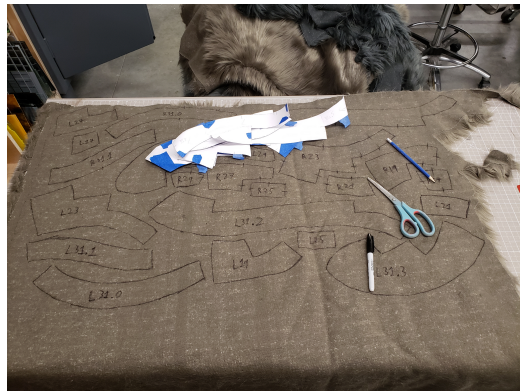


Fig. 1. Exterior of the Protogen Head (with the initial visor removed)

Fursuits are Planar Graphs



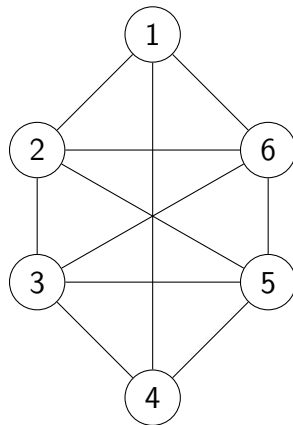
Bunsen's head pattern



Unsigned Long's tail pattern

Fursuits are Planar Graphs

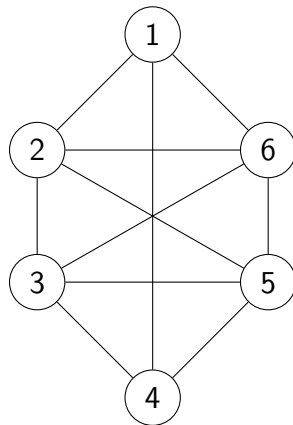
- Graph:
 - Vertices labeled $1, 2, \dots, n$
 - Edges go between two vertices
 - At most $n \cdot (n - 1) / 2$ edges^a



^aAssuming simple graphs

Fursuits are Planar Graphs

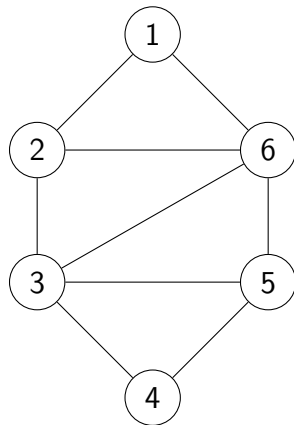
- Graph:
 - Vertices labeled $1, 2, \dots, n$
 - Edges go between two vertices
 - At most $n \cdot (n - 1) / 2 = \underline{O(n^2)}$ edges^a



^aAssuming simple graphs

Fursuits are Planar Graphs

- Graph:
 - Vertices labeled $1, 2, \dots, n$
 - Edges go between two vertices
 - At most $n \cdot (n - 1) / 2 = \underline{O(n^2)}$ edges^a
- Planar graphs: graphs that can be drawn without crossing edges
 - At most $3n - 6 = \underline{O(n)}$ edges!^b
 - Fursuits are planar graphs
 - Seam count = $O(\text{Fabric patch count})$

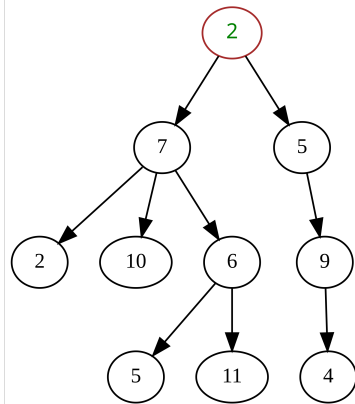


^aAssuming simple graphs

^bAssuming faces have degree ≥ 3

Trees!

- Trees are the building block of most data structures.
- Every node has one parent, except for the root which has no parent.
- Nodes with no children are leaves, nodes with children are internal nodes.



By Paddy3118 - Own work, CC BY-SA 4.0,

<https://commons.wikimedia.org/w/index.php?curid=83223854>

A-Z of Trees (non-exhaustive)

AVL Trees

k-d Tree

B Trees

Link-Cut Tree

Cartesian Trees

Merkle Trees

Universal B Trees

Decision Tree

N Tree

Vantage Point Tree

Exponential Trees

Order Statistics Tree

Wallace Tree

Fenwick Trees

PQ Tree

X-fast Trie

Gomory-Hu Tree

Quadtree

Y-fast Trie

H-Tree

Red Black Trees

Zip Trees

Interval Tree

Splay Trees

Judy Array

Tango Trees

Subsections Subtrees

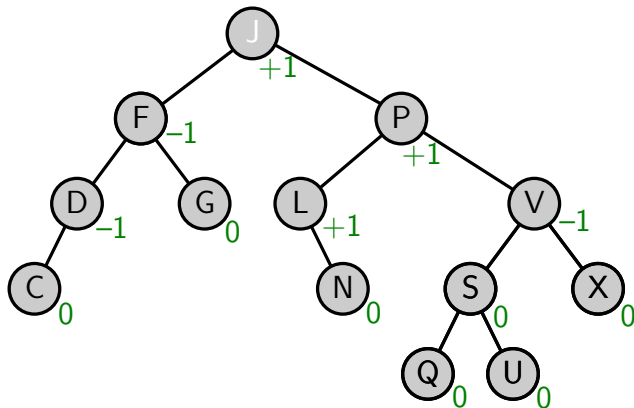
- ARSTZ : Balanced Binary Search Trees (BST)
- BCMUXY : Other Binary Trees
- IKQV : Spatial Data Structures
- EFGJLP : Niche data structures
- DHNOW : Non-data structures

Subsections Subtrees

- **ARSTZ** : Balanced Binary Search Trees (BST)
- **BCMUXY** : Other Binary Trees
- **IKQV** : Spatial Data Structures
- **EFGJLP** : Niche data structures
- **DHNOW** : Non-data structures

AVL Trees¹

- First $O(\log n)$ self-balancing binary search tree!
- Ensures the height of its children do not differ by > 1

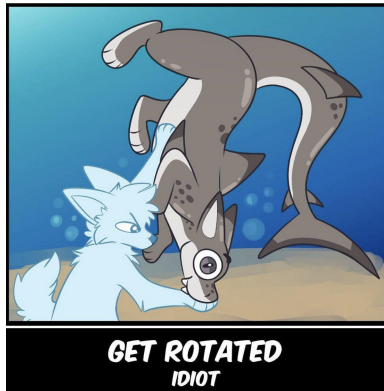


<https://commons.wikimedia.org/w/index.php?curid=49182185>

¹Adelson-Velsky, Georgy; Landis, Evgenii (1962)

AVL Trees (Rotation)

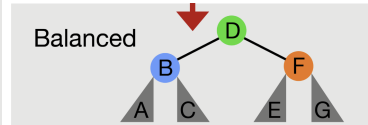
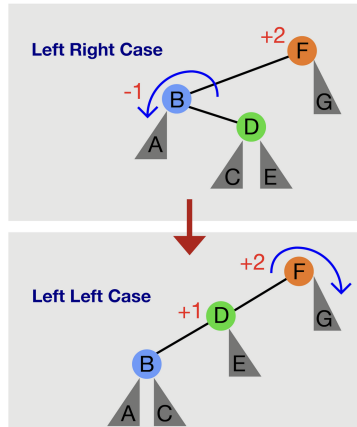
- Imbalances are fixed by rotations, fundamental local operations for many balanced BSTs



[/r/furry_irl/comments/1dumfg3/](https://www.reddit.com/r/furry_irl/comments/1dumfg3/)

AVL Trees (Rotation)

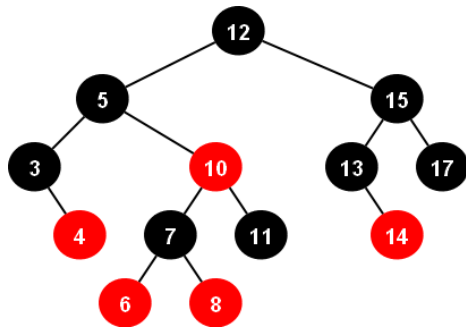
- Imbalances are fixed by rotations, fundamental local operations for many balanced BSTs



<https://pages.cs.wisc.edu/~qingyi/>

Red Black Tree²

- Balanced BST, red-black coloring chosen because it looked best on the laser printer they used.
- All root-to-leaf paths have the same number of black nodes
- No red node has a red parent



<https://pages.cs.wisc.edu/~wyoungjun/>

²Guibas, Leonidas J.; Sedgewick, Robert (1978).

Splay?

splay 1 of 3 verb

ˈsplā

splayed; splaying; splays

[Synonyms of splay](#) >

transitive verb

1 : to cause to spread outward

2 : to make oblique : **BEVEL**

intransitive verb

1 : to extend apart or outward especially in an awkward manner

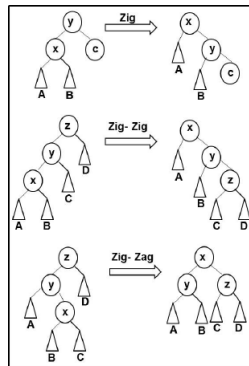
2 : **SLOPE, SLANT**



<https://en.wikifur.com/wiki/Furpile>

Splay Tree³

- After every operation, splay the searched node to the root of the tree using double-rotations.
- Suspected to be optimal for any sequence of BST operations up to a constant factor (Dynamic Optimality Conjecture)



Trabelsi, Zouheir & Zeidan, Safaa & Masud, Mehedy & Ghoudi, Kilani.

(2015). Statistical Dynamic Splay Tree Filters.

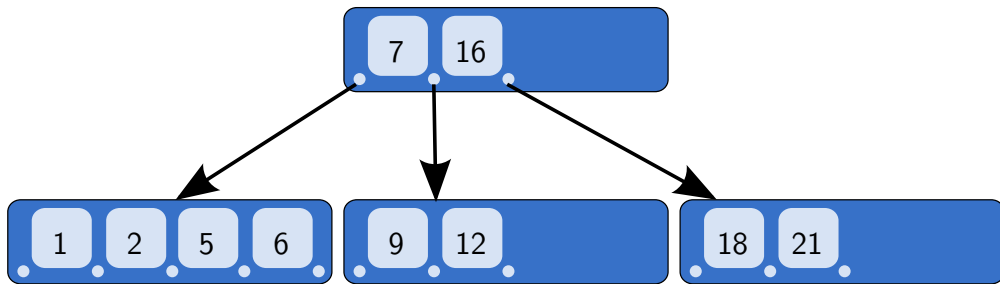
³Sleator, Daniel D.; Tarjan, Robert E. (1985).

B Tree⁴

- Generalising binary search trees to > 2 children
- Internal nodes hold both data and pointers to children nodes in sorted order
- Forms the basis of many filesystem structures
 - Apple HFS+, Microsoft NTFS, Linux ext4

⁴Bayer, R.; McCreight, E. (July 1970).

B Tree (Diagram)



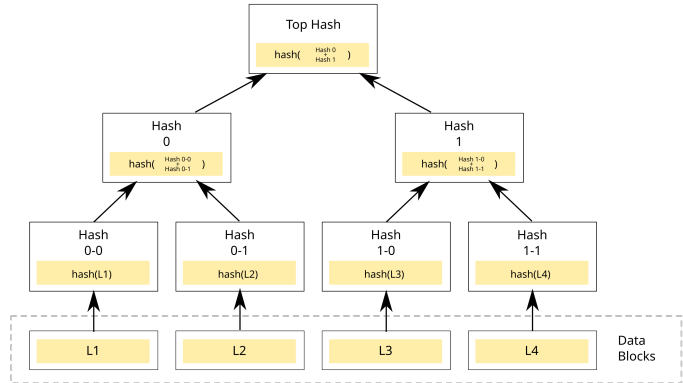
By CyHawk, CC BY-SA 3.0, <https://commons.wikimedia.org/w/index.php?curid=11701365>

What Rudy [Bayer] likes to say is, the more you think about what the B in B-Tree means, the better you understand B-Trees!

- Edward M. McCreight

Merkle Tree⁵

- Tree of hashes, widely used for data verification
 - Cryptographic schemes
 - P2P file sharing
 - Distributed filesystems
- Each internal node hashes the concatenated hashes of their children.
- Only recomputes $O(\log n)$ hashes when data changes!

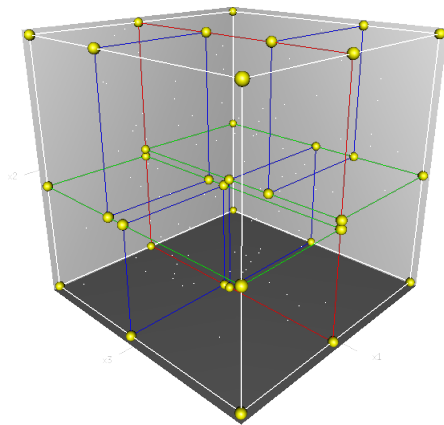


⁵Merkle, R. C. (1979)

k-d tree⁶

- Each node is a k -dimensional point, each internal node splits one of the dimensions at the point.
- $O(\log n)$ insert, delete and search for a point, fast^a range search and nearest neighbour search.

^asubject to curse of dimensionality



⁶Bentley, J. L. (1975).

Exponential Trees⁷

- Has niche uses in hash tables
- Time complexity is a *bit cursed*

Abstract

We present a significant improvement on linear space deterministic sorting and searching. On a unit-cost RAM with word size w , an ordered set of n w -bit keys (viewed as binary strings or integers) can be maintained in

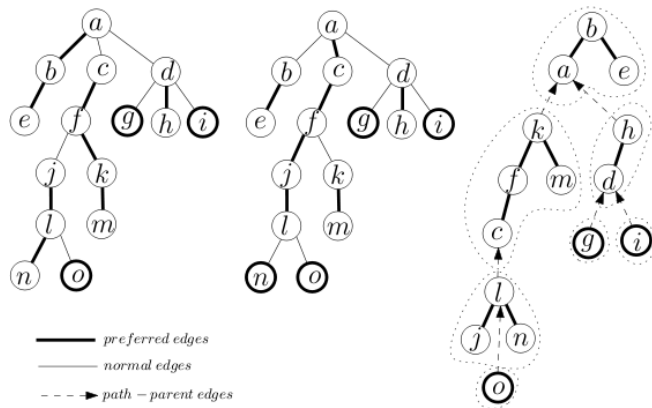
$$O \left(\min \left\{ \begin{array}{l} \sqrt{\log n} \\ \frac{\log n}{\log w} + \log \log n \\ \log w \log \log n \end{array} \right\} \right)$$

⁷Andersson, Arne (October 1996)



Link-Cut Tree⁸

- Maintains a set of rooted trees
- Amortized $O(\log n)$ link, cut and find-root at any node
- Improves Dinic's Algorithm (for max-flow) from $O(V^2E)$ to $O(VE \log V)$.

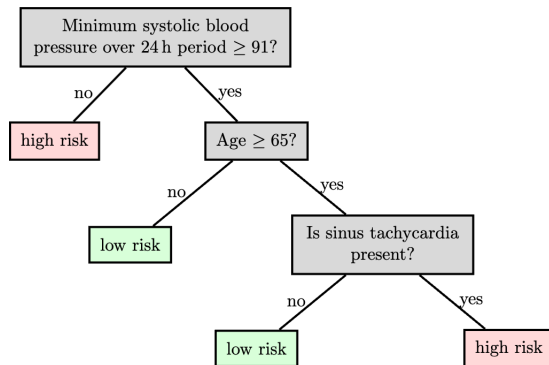


By Drrilll, CC BY-SA 3.0, <https://commons.wikimedia.org/w/index.php?curid=25495327>

⁸Sleator, D. D.; Tarjan, R. E. (1983).

Decision Trees

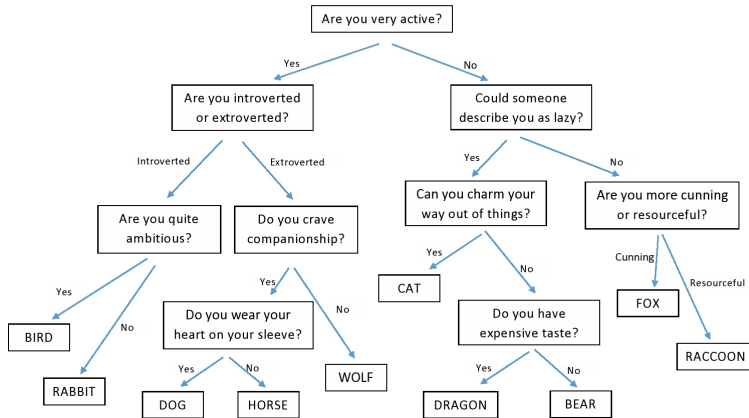
- Each node represents a query, each leaf represents a category/decision.
- Commonly used in decision analysis and machine learning
- Fuzzy Decision Trees are also a thing



MIT 6.390 Intro ML Course Notes and Breiman, Friedman, Olshen, Stone (1984)

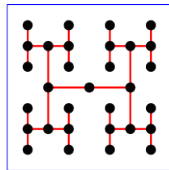
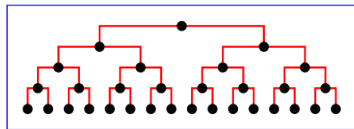
(Furry) Decision Trees (r/furry/comments/4gxm0l/)

What is your fursona?

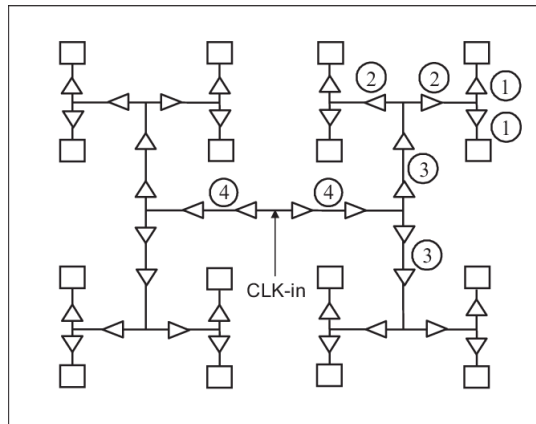


H Trees

- Binary tree fractal!
- Used in chip design in order to distribute wide reaching nets



<https://www.tamurajones.net/FractalGenealogy.xhtml>



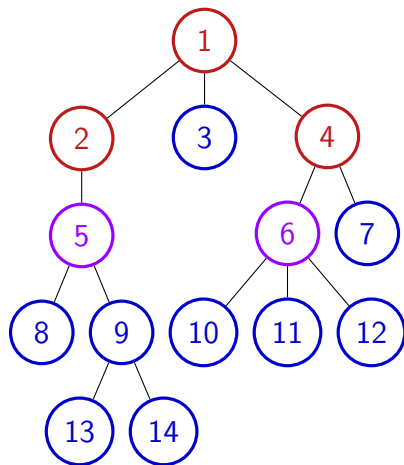
Tawfik, Sherif & Kursun, Volkan. (2008).

Missing N-tree

- Seriously I could not find any tree data structure that starts with N

Macro-Micro Tree Decomposition

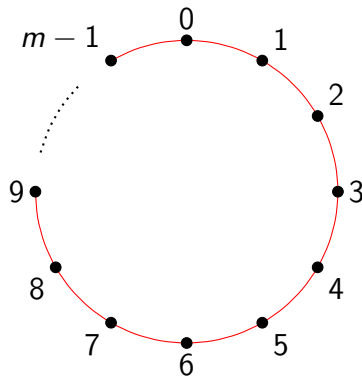
- Node is micro if it has less than $O(\log(n))$ descendants (else macro)
- Node is macro leaf if it is macro and all its children are micro
- Subtree is microtree if its parent is a macro leaf
- There are at most $O(n/\log(n))$ macro leaves
 - Macro leaves have $O(\log(n))$ descendants, and do not share descendants
- There are $O(n^{1/c})$ distinct microtree shapes
 - Tree shape count is exponential in node count
 - Number of microtree nodes is logarithmic in n
 - log and exp cancel out to $O(n^{1/c})$



Clock Arithmetic

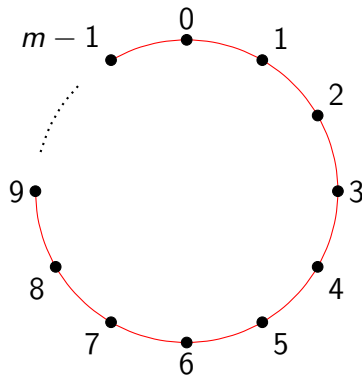
- Clocks wrap around after 24h^a
- Generalize by fixing a modulus m , and do all math as remainders when dividing by m
 - All integers $x = qm + r$ are treated as the same number
- When to compute remainders?
 - $(q_1m + r_1) + (q_2m + r_2) = (q_1 + q_2)m + r_1 + r_2.$
 - $(q_1m + r_1) \cdot (q_2m + r_2) = (q_1q_2m + q_1r_2 + r_1q_2)m + r_1r_2.$

^aIgnoring the fact that DST just ended



Clock Arithmetic

- Division by a ?
 - If a coprime with m , then $a \cdot a^{-1} = 1$ exists
- Exponentiation?
 - $1, a, a^2, \dots, a^x, \dots \pmod m$ will eventually repeat (pigeonhole)
 - If a^{-1} exists, first repetition happens at $a^n = 1$ (n is the order of a)
- This is number theory!
- Can't teach all of number theory in 1h
- But useful for cryptography, etc.



Fast Furrier Transform

- Fourier Transform?
 - Typically computed with complex numbers
 - Transform signal between time and frequency domain

Fast Furrier Transform

- Fourier Transform?
 - Typically computed with complex numbers
 - Transform signal between time and frequency domain
- This is not a math panel!
 - See this 3Blue1Brown video:
“But what is the Fourier Transform? A visual introduction.” →



Fast Furrier Transform

- Fourier Transform?
 - Typically computed with complex numbers
 - Transform signal between time and frequency domain
- This is not a math panel!
 - See this 3Blue1Brown video:
“But what is the Fourier Transform? A visual introduction.” →
- This is an algorithms panel :3
 - We will use FFT to multiply polynomials in $O(n \log n)$ time



Fast Furrier Transform: Polynomials

Diagram illustrating the components of a polynomial term:

coefficient (points to 2 in $2x$)

term (points to $-4x^2$)

degree (points to 3 in x^3)

$$f(x) = 3 + 2x - 4x^2 + x^3$$

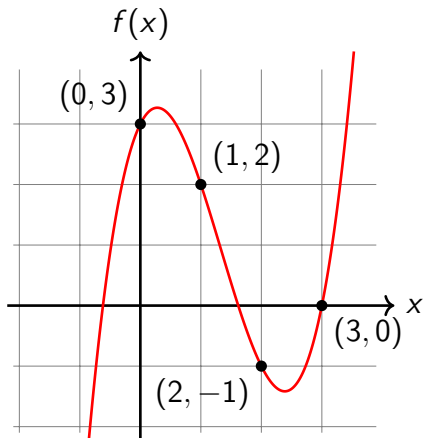
$$f(0) = 3$$

$$f(1) = 2$$

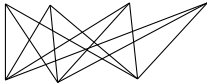
$$f(2) = -1$$

$$f(3) = 0$$

Evaluating a polynomial at a point
takes $O(n)$ time



Fast Furrier Transform: Multiplication

$$f(x) = 3 + 2x - 4x^2 + x^3$$


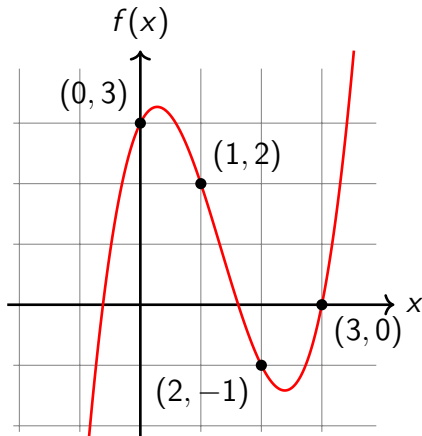
$$g(x) = 2 - x + x^2$$

$$h(x) = f(x) \cdot g(x) = 6 + x - 7x^2 + 8x^3 - 5x^4 + x^5$$

Multiplying two polynomials of degree n by distributing takes $O(n^2)$ time

Fast Furrier Transform: Polynomial Interpolation/Evaluation

- For a set of n points, there is a unique polynomial with degree less than n that passes through all the points
- Evaluating the polynomial at n different x values to find these points typically takes $O(n^2)$ time
- If we choose the x values cleverly, we can do better



Fast Furrier Transform: Evaluation at ± 1

$$f(x) = a_0 + a_1x + a_2x^2 + a_3x^3 + \cdots + a_{n-2}x^{n-2} + a_{n-1}x^{n-1}$$

Fast Furrier Transform: Evaluation at ± 1

$$f(x) = a_0 + a_1x + a_2x^2 + a_3x^3 + \cdots + a_{n-2}x^{n-2} + a_{n-1}x^{n-1}$$

$$f(1) = a_0 + a_1 + a_2 + a_3 + \cdots + a_{n-2} + a_{n-1}$$

$$f(-1) = a_0 - a_1 + a_2 - a_3 + \cdots + a_{n-2} - a_{n-1}$$

Fast Furrier Transform: Evaluation at ± 1

$$f(x) = a_0 + a_1x + a_2x^2 + a_3x^3 + \cdots + a_{n-2}x^{n-2} + a_{n-1}x^{n-1}$$

$$f(1) = a_0 + a_1 + a_2 + a_3 + \cdots + a_{n-2} + a_{n-1}$$

$$f(-1) = a_0 - a_1 + a_2 - a_3 + \cdots + a_{n-2} - a_{n-1}$$

$$f(1) = (a_0 + a_2 + \cdots + a_{n-2}) + (a_1 + a_3 + \cdots + a_{n-1})$$

$$f(-1) = (a_0 + a_2 + \cdots + a_{n-2}) - (a_1 + a_3 + \cdots + a_{n-1})$$

Fast Furrier Transform: Evaluation at ± 1

$$f(x) = a_0 + a_1x + a_2x^2 + a_3x^3 + \cdots + a_{n-2}x^{n-2} + a_{n-1}x^{n-1}$$

$$f(1) = a_0 + a_1 + a_2 + a_3 + \cdots + a_{n-2} + a_{n-1}$$

$$f(-1) = a_0 - a_1 + a_2 - a_3 + \cdots + a_{n-2} - a_{n-1}$$

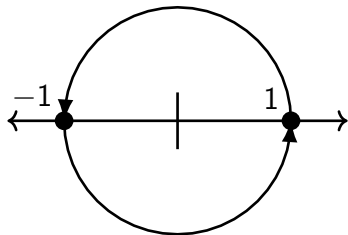
$$f(1) = (a_0 + a_2 + \cdots + a_{n-2}) + (a_1 + a_3 + \cdots + a_{n-1})$$

$$f(-1) = (a_0 + a_2 + \cdots + a_{n-2}) - (a_1 + a_3 + \cdots + a_{n-1})$$

Number of operations is halved!

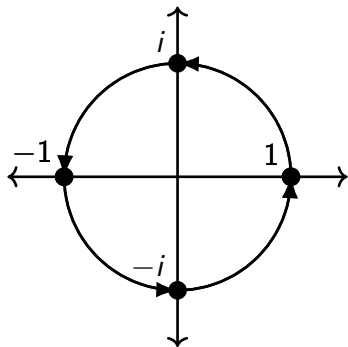
Fast Furrier Transform: Roots of Unity

- The even/odd trick works because repeatedly multiplying by -1 cycles between -1 and 1
- It'd be convenient if we had other values making this kind of cycle, and if these cycles were longer



Fast Furrier Transform: Roots of Unity

- The even/odd trick works because repeatedly multiplying by -1 cycles between -1 and 1
- It'd be convenient if we had other values making this kind of cycle, and if these cycles were longer
- Complex numbers give us values r where $r^n = 1$
- These are called roots of unity
- FFT evaluates f at $1, r, r^2, \dots, r^{n-1}$



Fast Furrier Transform: Odd/Even Trick

$$f(x) = a_0 + a_1x + a_2x^2 + \cdots + a_{n-1}x^{n-1}$$

Fast Furrier Transform: Odd/Even Trick

$$f(x) = a_0 + a_1x + a_2x^2 + \cdots + a_{n-1}x^{n-1}$$

$$f_e(x) = a_0 + a_2x^2 + a_4x^4 + \cdots + a_{n-2}x^{n-2}$$

$$f_o(x) = a_1x + a_3x^3 + a_5x^5 + \cdots + a_{n-1}x^{n-1}$$

Fast Furrier Transform: Odd/Even Trick

$$f(x) = a_0 + a_1x + a_2x^2 + \cdots + a_{n-1}x^{n-1}$$

$$f_e(x) = a_0 + a_2x^2 + a_4x^4 + \cdots + a_{n-2}x^{n-2}$$

$$f_o(x) = a_1x + a_3x^3 + a_5x^5 + \cdots + a_{n-1}x^{n-1}$$

Fast Furrier Transform: Odd/Even Trick

$$f(x) = a_0 + a_1x + a_2x^2 + \cdots + a_{n-1}x^{n-1}$$

$$f_e(x) = a_0 + a_2x^2 + a_4x^4 + \cdots + a_{n-2}x^{n-2}$$

$$f_o(x) = a_1x + a_3x^3 + a_5x^5 + \cdots + a_{n-1}x^{n-1}$$

$$f_o(x) = x(a_1 + a_3x^2 + a_5x^4 + \cdots + a_{n-1}x^{n-2})$$

Fast Furrier Transform: Odd/Even Trick

$$f(x) = a_0 + a_1x + a_2x^2 + \cdots + a_{n-1}x^{n-1}$$

$$f_e(x) = a_0 + a_2x^2 + a_4x^4 + \cdots + a_{n-2}x^{n-2}$$

$$f_o(x) = a_1x + a_3x^3 + a_5x^5 + \cdots + a_{n-1}x^{n-1}$$

$$f_o(x) = x(a_1 + a_3x^2 + a_5x^4 + \cdots + a_{n-1}x^{n-2})$$

Fast Furrier Transform: Odd/Even Trick

$$f(x) = a_0 + a_1x + a_2x^2 + \cdots + a_{n-1}x^{n-1}$$

$$f_e(x) = a_0 + a_2x^2 + a_4x^4 + \cdots + a_{n-2}x^{n-2}$$

$$f_o(x) = a_1x + a_3x^3 + a_5x^5 + \cdots + a_{n-1}x^{n-1}$$

$$f_o(x) = x(a_1 + a_3x^2 + a_5x^4 + \cdots + a_{n-1}x^{n-2})$$

$$g_e(y) = a_0 + a_2y + a_4y^2 + \cdots + a_{n-2}y^{n/2-1}$$

$$g_o(y) = a_1 + a_3y + a_5y^2 + \cdots + a_{n-1}y^{n/2-1}$$

$$f(x) = g_e(x^2) + xg_o(x^2)$$

Fast Furrier Transform: Divide and Conquer

$$f(x) = a_0 + a_1x + a_2x^2 + \dots + a_{n-1}x^{n-1} = g_e(x^2) + xg_o(x^2)$$

$$g_e(y) = a_0 + a_2y + a_4y^2 + \dots + a_{n-2}y^{n/2-1}$$

$$g_o(y) = a_1 + a_3y + a_5y^2 + \dots + a_{n-1}y^{n/2-1}$$

- We need to evaluate g_e and g_o at $y = x^2 =$

$$(1)^2, \quad (r)^2, \quad (r^2)^2, \dots, \quad (r^{n/2-1})^2, \quad (r^{n/2})^2, \quad (r^{n/2+1})^2, \dots, \quad (r^{n-1})^2$$

Fast Furrier Transform: Divide and Conquer

$$f(x) = a_0 + a_1x + a_2x^2 + \dots + a_{n-1}x^{n-1} = g_e(x^2) + xg_o(x^2)$$

$$g_e(y) = a_0 + a_2y + a_4y^2 + \dots + a_{n-2}y^{n/2-1}$$

$$g_o(y) = a_1 + a_3y + a_5y^2 + \dots + a_{n-1}y^{n/2-1}$$

- We need to evaluate g_e and g_o at $y = x^2 =$

$$\begin{array}{ccccccc} (1)^2, & (r)^2, & (r^2)^2, \dots, & (r^{n/2-1})^2, & (r^{n/2})^2, & (r^{n/2+1})^2, \dots, & (r^{n-1})^2 \\ 1, & r^2, & r^4, \dots, & r^{n-2}, & r^n = 1, & r^2, \dots, & r^{n-2} \end{array}$$

Fast Furrier Transform: Divide and Conquer

$$f(x) = a_0 + a_1x + a_2x^2 + \dots + a_{n-1}x^{n-1} = g_e(x^2) + xg_o(x^2)$$

$$g_e(y) = a_0 + a_2y + a_4y^2 + \dots + a_{n-2}y^{n/2-1}$$

$$g_o(y) = a_1 + a_3y + a_5y^2 + \dots + a_{n-1}y^{n/2-1}$$

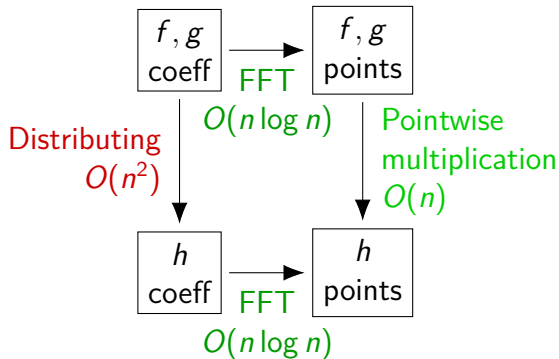
- We need to evaluate g_e and g_o at $y = x^2 =$

$$\begin{array}{ccccccc} (1)^2, & (r)^2, & (r^2)^2, \dots, & (r^{n/2-1})^2, & (r^{n/2})^2, & (r^{n/2+1})^2, \dots, & (r^{n-1})^2 \\ 1, & r^2, & r^4, \dots, & r^{n-2}, & r^n = 1, & r^2, \dots, & r^{n-2} \end{array}$$

- To evaluate f at n roots of unity, we evaluate g_e and g_o at $n/2$ roots of unity
- Recursion! $T(n) = 2T(n/2) + O(n) = O(n \log n)$

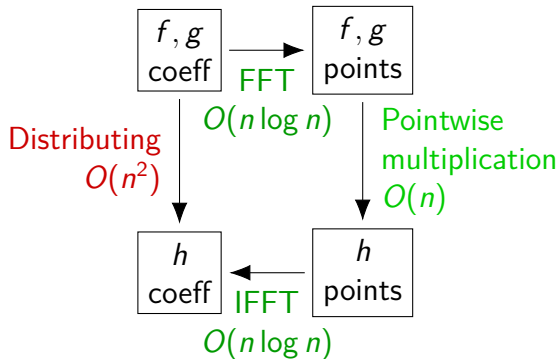
Fast Furrier Transform: Inverse FFT?

- Multiplying polynomials of degree n
 - Distributing would take $O(n^2)$
 - FFT takes $O(n \log n)$
 - Pointwise multiplication takes $O(n)$



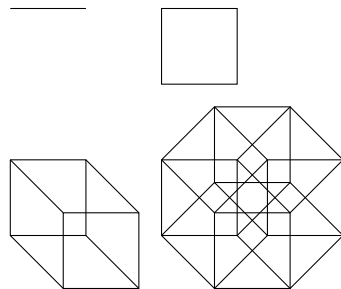
Fast Furrier Transform: Inverse FFT?

- Multiplying polynomials of degree n
 - Distributing would take $O(n^2)$
 - FFT takes $O(n \log n)$
 - Pointwise multiplication takes $O(n)$
- FFT is (approximately) its own inverse
 - No proof here: not a math panel
 - IFFT takes $O(n \log n)$
 - The Anti-Furry Transform is just a Furry Transform in disguise :3

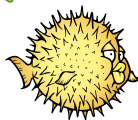


Curse of Dimensionality

- As number of dimensions increases, it's easier for points to be further apart
- Noisy dimensions obscure useful information
- Ex: For a hypercube in d dimensions with side length 1, furthest points are $\sqrt{d}$ away.
- Many algos are exponential in input dimension
 - k -d tree
 - Convex hull



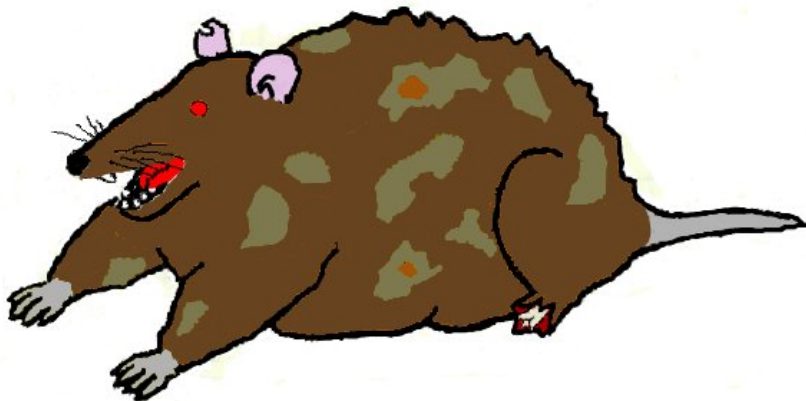
Animal Computing Mascots



Animal Computing Mascots: Trans Rights!



Animal Computing Mascots: Unofficial Mascot of C++



Animal Computing Mascots: Powershell...



Animal Computing Mascots: WSL

From: Richard Stallman
Subject: WSL
Date: Mon, 23 Jan 2023 22:50:01 -0500

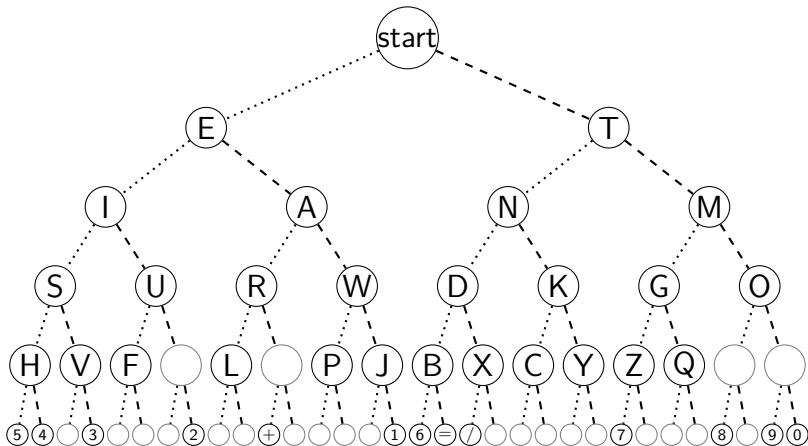
```
[[[ To any NSA and FBI agents reading my email: please consider   ]]]  
[[[ whether defending the US Constitution against all enemies,      ]]]  
[[[ foreign or domestic, requires you to follow Snowden's example. ]]]
```

How about pronouncing (and writing) "WSL" as "weasel"?

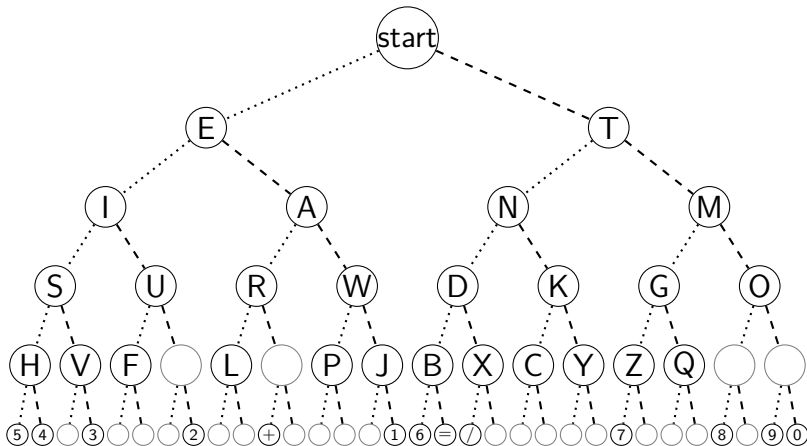
--

Dr Richard Stallman (<https://stallman.org>)
Chief GNUisance of the GNU Project (<https://gnu.org>)
Founder, Free Software Foundation (<https://fsf.org>)
Internet Hall-of-Famer (<https://internethalloffame.org>)

Morse Code

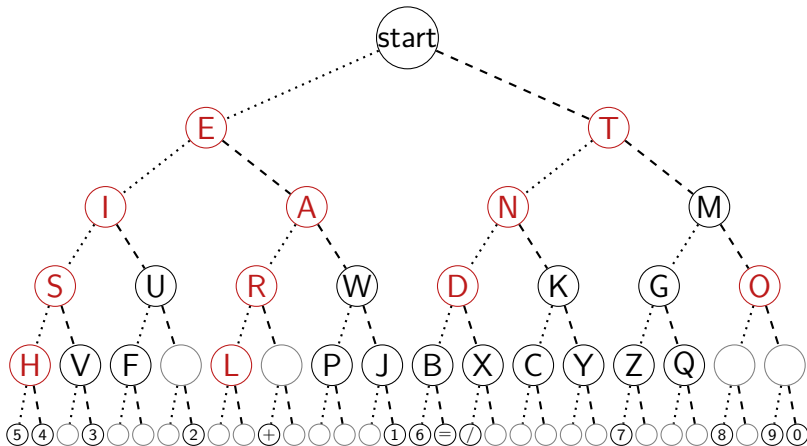


Morse Code



Letter	Frequency
E	12.7%
T	9.1%
A	8.2%
O	7.5%
I	7.0%
N	6.7%
S	6.3%
H	6.1%
R	6.0%
D	4.3%
L	4.0%

Morse Code



Letter	Frequency
E	12.7%
T	9.1%
A	8.2%
O	7.5%
I	7.0%
N	6.7%
S	6.3%
H	6.1%
R	6.0%
D	4.3%
L	4.0%

Braille

Aroga
TECHNOLOGIES

Unified English Braille Chart

ALPHABET AND NUMBERS	PUNCTUATION	SIGNS OF OPERATION AND COMPARISON	ALPHABETIC WORDSIGNS	STRONG GROUPSIGNS	INITIAL-LETTER CONTRACTIONS	FINAL-LETTER GROUPSIGNS	SHORTFORM WORDS	
1 2 3 4 5 6 7 8 9 0 a b c d e f g h i j k l m n o p q r s t u v x y z w	comma , period . apostrophe ' . colon : dash - long dash — exclamation mark ! hyphen - question mark ? semicolon ; ellipsis ... forward slash / backward slash \ opening outer quotation mark " " " " closing outer quotation mark " " " " opening inner quotation mark " " " " closing inner quotation mark " " " " closing inner quotation mark " " " "	plus + minus - multiplication x multiplication dot · division ÷ greater than > less than < equals = cent ¢ dollar \$ euro € British Pound £ feet ' f inches " i percent % degree ° angle ∠ hashtag # ampersand & copyright © trademark ™ superscript indicator subscript indicator bullet • @ sign @ asterisk * dot locator for mention	b but c can d do e every f from g go h have i just k knowledge l like m more n not p people q quite r rather s so t that u us v very w will x it y you z as	ch sh th wh ou at gh ed er ow ar ing ea bb cc ff	day ever father here less know loud mother name one part question right some time under work young	ound ance sion less oust already ence ong ful tion ment ity	ab about abv above ac across af after afn afternoon afw afterward ag again aga against alm almost alr already al also alth although alt altogether alw always bec because bef before beh behind bel below ben beneath bes beside bet between bey beyond bli blind brl brilliant chl children con conceive con-v conceiving cou could dec decide dec-v deceiving del declare dclg declaring ei either fir first fr from gd good grt great	hgf himself hmf himself imn immediate its its itf itself ltr letter lit little mch much mst must mys myself nec necessary nei neither onf oneself our ourselves pd paid per perceive per-v perceiving perh perhaps qck quick rec receive rec-v receiving rej rejoice rej-v rejoicing sd said shd should sh such thm themselves thrt thrift tod today tog together tom tomorrow ton tonight wd would yr your yrs yourself you yourselves
INDICATORS	CURRENCY AND MEASUREMENT	SPECIAL SYMBOLS	STRONG CONTRACTIONS (Part and Whole Word)	LOWER GROUPSIGNS	Retired Contractions (not used in UEB)			
Numeric Capital letter word passage capital terminator Grade 1 word passage Grade 1 terminator Typeform italic symbol italic word italic passage italic terminator bold symbol bold word bold passage bold terminator underline symbol underline word underline passage underline terminator script symbol script word script passage script terminator	opening round parenthesis () closing round parenthesis) opening square bracket [] closing square bracket] opening curly bracket { } closing curly bracket } opening angle bracket < > closing angle bracket > <	percent % degree ° angle ∠ hashtag # ampersand & copyright © trademark ™ superscript indicator subscript indicator bullet • @ sign @ asterisk * dot locator for mention	and for of the with	child shall this were which out still	there character through where ought upon word these those whose cannot had many spirit world about	ble dd into ation com by ally to o'clock	brought to you by Aroga TECHNOLOGIES Visit our online store at www.aroga.com © Aroga Technologies 2014	

Braille

ALPHABET AND NUMBERS

1	2	3	4	5	6	7	8	9	0
a	b	c	d	e	f	g	h	i	j
⠁	⠃	⠉	⠙	⠑	⠋	⠗	⠕	⠎	⠚
k	l	m	n	o	p	q	r	s	t
⠅	⠇	⠓	⠝	⠥	⠏	⠖	⠞	⠡	⠤
u	v	x	y	z					w
⠥	⠦	⠭	⠽	⠵					⠠
⠠	⠡	⠢	⠣	⠤					⠠

Braille

ALPHABET AND NUMBERS

1	2	3	4	5	6	7	8	9	0
a	b	c	d	e	f	g	h	i	j
⠠	⠠	⠠	⠠	⠠	⠠	⠠	⠠	⠠	⠠
k	l	m	n	o	p	q	r	s	t
⠠	⠠	⠠	⠠	⠠	⠠	⠠	⠠	⠠	⠠
u	v	x	y	z					w
⠠	⠠	⠠	⠠	⠠					⠠

STRONG GROUPSIGNS

⠠	ch	⠠	gh
⠠	sh	⠠	ed
⠠	th	⠠	er
⠠	wh	⠠	ow
⠠	ou	⠠	ar
⠠	st	⠠	ing

Braille

ALPHABET AND NUMBERS

1	2	3	4	5	6	7	8	9	0
a	b	c	d	e	f	g	h	i	j
⠠	⠡	⠢	⠣	⠤	⠥	⠦	⠧	⠨	⠩
k	l	m	n	o	p	q	r	s	t
⠪	⠬	⠭	⠮	⠏	⠱	⠲	⠳	⠴	⠵
u	v	x	y	z					w
⠺	⠻	⠼	⠽	⠿					⠽

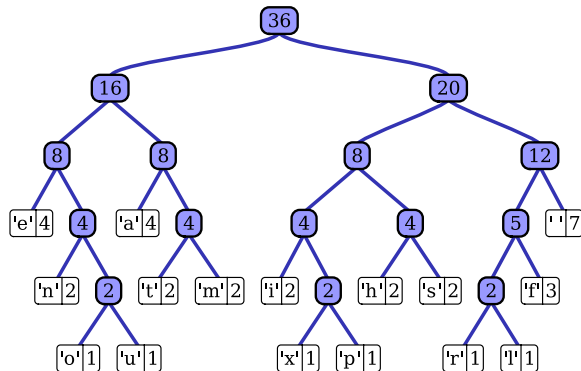
STRONG GROUPSIGNS

⠠	ch	⠠	gh
⠡	sh	⠡	ed
⠢	th	⠢	er
⠣	wh	⠣	ow
⠤	ou	⠤	ar
⠥	st	⠥	ing

“owo” is ⠏⠺⠺

Huffman Coding

- More frequent characters get shorter encodings
- Used in .zip files and other compression algorithms
- Won't go over details due to time

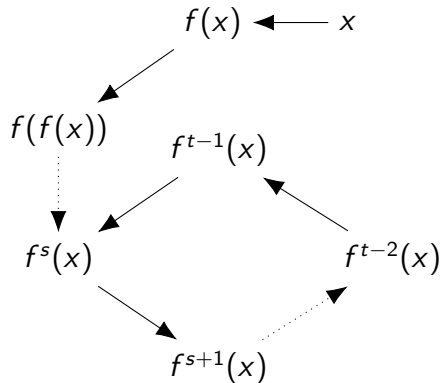


Tortoise and Hare: The Cycle Finding Problem

- Take some function $f : [n] \rightarrow [n]$, and some starting value x .
 - Notation: $[n]$ is the numbers $\{1, \dots, n\}$, and f is a function that takes a number from $[n]$, and outputs a number in $[n]$

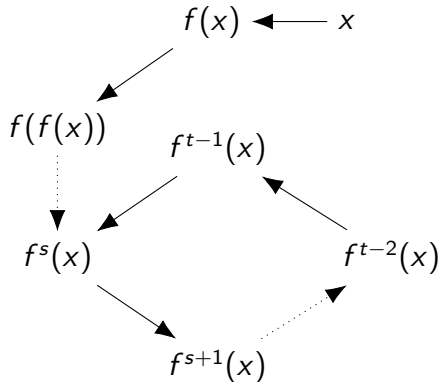
Tortoise and Hare: The Cycle Finding Problem

- Take some function $f : [n] \rightarrow [n]$, and some starting value x .
 - Notation: $[n]$ is the numbers $\{1, \dots, n\}$, and f is a function that takes a number from $[n]$, and outputs a number in $[n]$
- By pidgeonhole principle, the sequence $x, f(x), f(f(x)), \dots, f^i(x), \dots$ will repeat



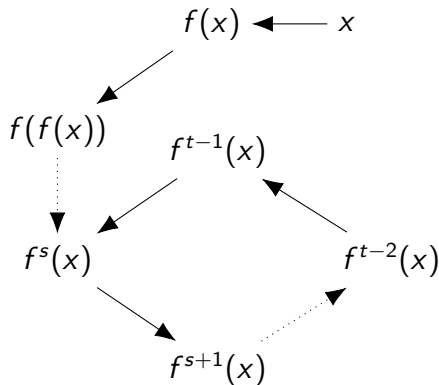
Tortoise and Hare: The Cycle Finding Problem

- Take some function $f : [n] \rightarrow [n]$, and some starting value x .
 - Notation: $[n]$ is the numbers $\{1, \dots, n\}$, and f is a function that takes a number from $[n]$, and outputs a number in $[n]$
- By pidgeonhole principle, the sequence $x, f(x), f(f(x)), \dots, f^i(x), \dots$ will repeat
- Find s, t such that $f^s(x) = f^{s+t}(x)$



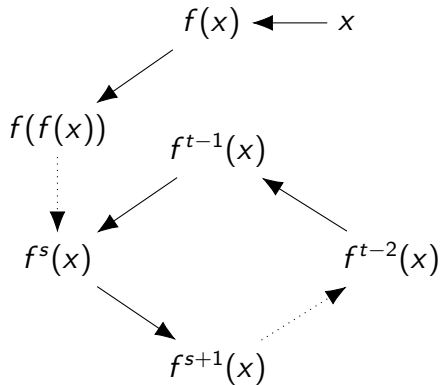
Tortoise and Hare: Floyd's Algorithm

- Algorithm description
 - Tortoise and hare start with $x_t := x$ and $x_h := x$.
 - When tortoise computes $x_t := f(x_t)$, hare computes $x_h := f(f(x_h))$.



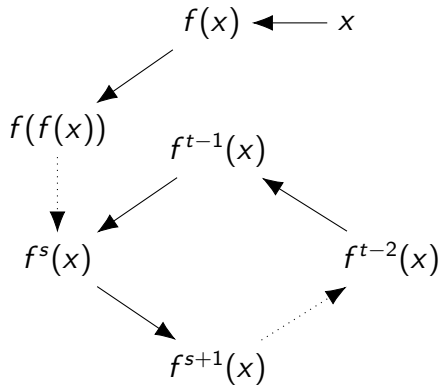
Tortoise and Hare: Floyd's Algorithm

- Algorithm description
 - Tortoise and hare start with $x_t := x$ and $x_h := x$.
 - When tortoise computes $x_t := f(x_t)$, hare computes $x_h := f(f(x_h))$.
- Algorithm analysis
 - Eventually both tortoise and hare will enter the cycle
 - When both are in the cycle, the hare will catch up to the tortoise



Tortoise and Hare: Floyd's Algorithm

- Algorithm description
 - Tortoise and hare start with $x_t := x$ and $x_h := x$.
 - When tortoise computes $x_t := f(x_t)$, hare computes $x_h := f(f(x_h))$.
- Algorithm analysis
 - Eventually both tortoise and hare will enter the cycle
 - When both are in the cycle, the hare will catch up to the tortoise
- Used in Pollard's rho algorithm



Tortoise and Hare: Pollard's Kangaroo

How Long Does it Take to Catch a Wild Kangaroo?

Ravi Montenegro *

Prasad Tetali †

November 7, 2010

Abstract

We develop probabilistic tools for upper and lower bounding the expected time until two independent random walks on $\mathbb{Z}$ intersect each other. This leads to the first sharp analysis of a non-trivial Birthday attack, proving that Pollard's Kangaroo method solves the discrete logarithm problem $g^x = h$ on a cyclic group in expected time $(2 + o(1))\sqrt{b - a}$ for an average $x \in [a, b]$. Our methods also resolve a conjecture of Pollard's, by showing that the same bound holds when step sizes are generalized from powers of 2 to powers of any fixed n .



Primal Duel

- Optimization problems
 - Find minimum/maximum of objective function
 - Variables must satisfy constraints

Primal Duel

- Optimization problems
 - Find minimum/maximum of objective function
 - Variables must satisfy constraints
- Initial formulation called the “primal”

Primal Duel

- Optimization problems
 - Find minimum/maximum of objective function
 - Variables must satisfy constraints
- Initial formulation called the “primal”
- There is a transformation that moves primal constraints into objective function terms, and moves primal objective function terms into constraints
 - The resulting formulation is called the “dual”

Primal Duel

- Optimization problems
 - Find minimum/maximum of objective function
 - Variables must satisfy constraints
- Initial formulation called the “primal”
- There is a transformation that moves primal constraints into objective function terms, and moves primal objective function terms into constraints
 - The resulting formulation is called the “dual”
- Some optimization algorithms switch between primal and dual formulations
 - Primal-dual methods

Data Structures & Algorithms @ Furpocalypse 2025

The Midnight Dimension

Bunsen Bitti, Unsigned Long

November 2, 2025

